

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 3

DML OPERATIONS AND DATA RETRIEVAL

1. Aim

To insert, update and delete data using DML commands and to retrieve data from tables using SELECT with WHERE, ORDER BY and LIMIT in MySQL.

2. Problem Statement

Populate the tables of *college_db* (created in Practical 2) with sample data for five departments, eight instructors, ten students, ten courses and twenty-five enrollments, and answer the following:

1. Display all departments, and each student's roll number, name in capitals and age.
2. List AIML students from Pune; instructors earning more than 1,50,000 or working as Associate Professor; courses not offered by department 10.
3. List enrollments with marks between 80 and 90; students from Mumbai, Nashik or Nagpur; students whose name starts with 'A' or who belong to AIML (roll number S24AI...); students without a city and courses without an instructor.
4. Sort students by department and youngest first; show the top five marks; show the second page (rows 4-6) of instructors ordered by salary.
5. Update the missing city of a student, give a 10% raise to Assistant Professors and add 5 grace marks to failing EC201 students and recompute their grade.
6. Delete a phone number, delete a student who has left the college (with enrollments) and attempt to delete a course that is still referenced.

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To insert data using the different forms of the INSERT statement.
2. To retrieve data using SELECT with column lists, aliases, expressions and DISTINCT.
3. To filter rows using WHERE with comparison, logical and special operators.
4. To sort and limit results using ORDER BY and LIMIT / OFFSET.
5. To modify and remove data using UPDATE and DELETE safely.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files

5. Theory

5.1 Data Manipulation Language (DML)

DML statements operate on the *data* stored in tables, unlike DDL which changes the structure. The four DML statements are INSERT, SELECT, UPDATE and DELETE (often called CRUD: Create, Read, Update, Delete). In MySQL, DML changes made with autocommit ON are made permanent immediately; inside a transaction they can be rolled back (Practical 10).

5.2 INSERT

```
INSERT INTO table_name VALUES (v1, v2, ...);           -- all columns, in order
INSERT INTO table_name (c1, c2) VALUES (v1, v2);      -- named columns
INSERT INTO table_name (c1, c2) VALUES (v1, v2), (v3, v4); -- multiple rows
INSERT INTO table_name (c1, c2) SELECT ... ;           -- rows from a query
```

Columns that are omitted receive their DEFAULT value, NULL, or the next AUTO_INCREMENT number. Every inserted row must satisfy all constraints of the table.

5.3 SELECT and Filtering with WHERE

```
SELECT [DISTINCT] column_list | *
FROM   table_name
[WHERE condition]
[ORDER BY column [ASC | DESC], ...]
[LIMIT row_count [OFFSET offset]];
```

Operator	Meaning	Example
=, <, <=, >, >=	Comparison	marks >= 60
AND, OR, NOT	Combine or negate conditions	city = 'Pune' AND dept_id = 20
BETWEEN a AND b	Inclusive range	marks BETWEEN 80 AND 90
IN (list)	Matches any value in the list	city IN ('Mumbai', 'Nashik')
LIKE pattern	% = any sequence, _ = exactly one character	student_name LIKE 'A%'
IS [NOT] NULL	Test for missing value (= NULL never matches)	city IS NULL

The logical order of evaluation of a SELECT statement is FROM, WHERE, SELECT, ORDER BY and finally LIMIT. Therefore a column alias can be used in ORDER BY but not in WHERE. The order of rows returned without ORDER BY is not guaranteed.

5.4 UPDATE and DELETE

```
UPDATE table_name SET col1 = expr1, col2 = expr2 [WHERE condition];
DELETE FROM table_name [WHERE condition];
```

Without a WHERE clause, UPDATE and DELETE affect **every** row of the table. The MySQL client option `--safe-updates` (SQL_SAFE_UPDATES = 1, enabled by default in MySQL Workbench) rejects such statements unless the WHERE clause uses a key column or a LIMIT is given. DELETE removes rows one by one and can be rolled back; TRUNCATE (DDL) removes all rows at once and resets AUTO_INCREMENT.

5.5 MySQL-Specific Syntax Used

- LIMIT n OFFSET m (or LIMIT m, n) - SQL Server uses TOP / OFFSET ... FETCH and Oracle uses FETCH FIRST n ROWS ONLY.
- Back-quotes (`Roll No`) quote identifiers containing spaces; standard SQL uses double quotes.
- TIMESTAMPDIFF(YEAR, dob, date) computes completed years (age) between two dates.

6. Procedure / Algorithm

1. Create *college_db* with the tables and constraints defined in Practical 2.
2. Insert data into the parent tables first (department, then instructor), followed by student, student_phone, course and enrollment, using single-row, column-list and multi-row INSERT.
3. Verify the number of rows inserted in each table.
4. Retrieve data with SELECT using *, selected columns, aliases, computed columns and DISTINCT.
5. Filter rows with WHERE using comparison, logical and special operators (BETWEEN, IN, LIKE, IS NULL).
6. Sort the result with ORDER BY on one or more columns and restrict it with LIMIT / OFFSET.
7. Modify data with UPDATE, displaying the rows before and after the change.
8. Remove data with DELETE and observe the effect of foreign key actions.
9. Record the output of every statement.

7. SQL Queries / Program

Q1: Create the database and tables (schema of Practical 2)

```
DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id      INT          NOT NULL,
    dept_code    VARCHAR(10)  NOT NULL,
    dept_name    VARCHAR(40)  NOT NULL,
    building     VARCHAR(20),
```

```

        budget      DECIMAL(12,2)  DEFAULT 1000000.00,
        CONSTRAINT pk_department PRIMARY KEY (dept_id),
        CONSTRAINT uq_dept_code  UNIQUE (dept_code),
        CONSTRAINT chk_budget    CHECK (budget > 0)
    );

CREATE TABLE instructor (
    instr_id      INT             NOT NULL,
    instr_name    VARCHAR(40)     NOT NULL,
    designation    VARCHAR(20)    NOT NULL,
    dept_id       INT,
    salary        DECIMAL(10,2)  NOT NULL,
    email         VARCHAR(40)     NOT NULL,
    mentor_id     INT,
    CONSTRAINT pk_instructor PRIMARY KEY (instr_id),
    CONSTRAINT uq_instr_email UNIQUE (email),
    CONSTRAINT chk_salary CHECK (salary BETWEEN 30000 AND 300000),
    CONSTRAINT fk_instr_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_instr_mentor FOREIGN KEY (mentor_id)
        REFERENCES instructor (instr_id) ON DELETE SET NULL
    );

CREATE TABLE student (
    student_id    INT             NOT NULL AUTO_INCREMENT,
    roll_no       VARCHAR(10)     NOT NULL,
    student_name  VARCHAR(40)     NOT NULL,
    gender        CHAR(1)         NOT NULL,
    dob           DATE            NOT NULL,
    dept_id       INT             NOT NULL,
    city          VARCHAR(30)     DEFAULT 'Pune',
    email         VARCHAR(40)     NOT NULL,
    CONSTRAINT pk_student PRIMARY KEY (student_id),
    CONSTRAINT uq_roll_no UNIQUE (roll_no),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT chk_gender CHECK (gender IN ('M', 'F', 'O')),
    CONSTRAINT chk_dob CHECK (dob >= '1995-01-01'),
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON UPDATE CASCADE
    );

CREATE TABLE student_phone (
    student_id    INT             NOT NULL,
    phone         CHAR(10)        NOT NULL,
    CONSTRAINT pk_student_phone PRIMARY KEY (student_id, phone),
    CONSTRAINT chk_phone CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_phone_student FOREIGN KEY (student_id)
        REFERENCES student (student_id) ON DELETE CASCADE
    );

CREATE TABLE course (
    course_id     CHAR(5)         NOT NULL,
    title         VARCHAR(40)     NOT NULL,
    dept_id       INT             NOT NULL,
    credits       TINYINT         NOT NULL,
    instr_id      INT,
    prereq_id     CHAR(5),
    CONSTRAINT pk_course PRIMARY KEY (course_id),
    CONSTRAINT uq_course_title UNIQUE (title),
    CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 5),
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id)
        REFERENCES department (dept_id) ON UPDATE CASCADE,
    CONSTRAINT fk_course_instr FOREIGN KEY (instr_id)
        REFERENCES instructor (instr_id) ON DELETE SET NULL,
    CONSTRAINT fk_course_prereq FOREIGN KEY (prereq_id)
        REFERENCES course (course_id)
    );

CREATE TABLE enrollment (

```

```

student_id INT NOT NULL,
course_id CHAR(5) NOT NULL,
semester TINYINT NOT NULL,
marks INT,
grade CHAR(2),
CONSTRAINT pk_enrollment PRIMARY KEY (student_id, course_id),
CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8),
CONSTRAINT chk_marks CHECK (marks BETWEEN 0 AND 100),
CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
REFERENCES student (student_id) ON DELETE CASCADE,
CONSTRAINT fk_enr_course FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

Q2: INSERT: single row, with column list, and multiple rows

```

INSERT INTO department VALUES (10, 'CSE', 'Computer Engineering', 'A Block', 2500000);
INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES
(20, 'AIML', 'AI and Machine Learning', 'B Block', 3000000),
(30, 'ECE', 'Electronics and Telecomm', 'C Block', 1800000),
(40, 'MECH', 'Mechanical Engineering', 'D Block', 1500000),
(50, 'CIVIL', 'Civil Engineering', 'D Block', 1200000);
INSERT INTO instructor VALUES
(101, 'Anil Kulkarni', 'Professor', 10, 185000, 'anil.k@example.com', NULL),
(102, 'Meera Joshi', 'Professor', 20, 172000, 'meera.j@example.com', 101),
(103, 'Rahul Patil', 'Asst. Prof', 10, 82000, 'rahul.p@example.com', 101),
(104, 'Sneha Deshmukh', 'Assoc. Prof', 20, 115000, 'sneha.d@example.com', 102),
(105, 'Vikram Shinde', 'Professor', 30, 168000, 'vikram.s@example.com', 101),
(106, 'Pooja Kale', 'Asst. Prof', 20, 78000, 'pooja.k@example.com', 104),
(107, 'Nitin Pawar', 'Asst. Prof', 50, 76000, 'nitin.p@example.com', 105),
(108, 'Kavita More', 'Assoc. Prof', 40, 112000, 'kavita.m@example.com', 105);
INSERT INTO student
(roll_no, student_name, gender, dob, dept_id, city, email) VALUES
('S24CS001', 'Aarav Sharma', 'M', '2005-03-14', 10, 'Pune', 'aarav@example.com'),
('S24CS002', 'Isha Gupta', 'F', '2005-07-22', 10, 'Mumbai', 'isha@example.com'),
('S24AI001', 'Rohan Mehta', 'M', '2005-01-09', 20, 'Pune', 'rohan@example.com'),
('S24AI002', 'Priya Nair', 'F', '2004-11-30', 20, 'Nashik', 'priya@example.com'),
('S24AI003', 'Kunal Verma', 'M', '2005-05-18', 20, 'Pune', 'kunal@example.com'),
('S24EC001', 'Sneha Iyer', 'F', '2005-09-02', 30, 'Nagpur', 'sneha@example.com'),
('S24EC002', 'Aditya Rao', 'M', '2004-12-25', 30, 'Mumbai', 'aditya@example.com'),
('S24ME001', 'Neha Kulkarni', 'F', '2005-02-11', 40, 'Pune', 'neha@example.com'),
('S24CS003', 'Yash Jadhav', 'M', '2005-08-05', 10, NULL, 'yash@example.com'),
('S24AI004', 'Ananya Desai', 'F', '2005-04-27', 20, 'Mumbai', 'ananya@example.com');
INSERT INTO student_phone VALUES
(1, '9876500011'), (1, '9876500012'), (2, '9876500021'), (3, '9876500031'),
(4, '9876500041'), (4, '9876500042'), (6, '9876500061'), (9, '9876500091');
INSERT INTO course VALUES
('CS101', 'Programming in C', 10, 4, 103, NULL),
('CS201', 'Data Structures', 10, 4, 103, 'CS101'),
('CS301', 'Database Management Systems', 10, 3, 101, 'CS201'),
('CS401', 'Advanced Databases', 10, 3, 101, 'CS301'),
('AI201', 'Python for Data Science', 20, 3, 104, 'CS101'),
('AI301', 'Machine Learning', 20, 4, 102, 'AI201'),
('AI401', 'Deep Learning', 20, 3, 106, 'AI301'),
('EC201', 'Digital Electronics', 30, 3, 105, NULL),
('EC301', 'Signals and Systems', 30, 4, NULL, 'EC201'),
('ME201', 'Engineering Mechanics', 40, 3, 108, NULL);
INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES
(1, 'CS101', 3, 78, 'A'), (1, 'CS201', 3, 85, 'A+'), (1, 'CS301', 4, 91, 'O'),
(1, 'AI201', 3, 72, 'A'), (2, 'CS101', 3, 88, 'A+'), (2, 'CS201', 3, 67, 'B+'),
(2, 'CS301', 4, 74, 'A'), (3, 'CS101', 3, 65, 'B+'), (3, 'AI201', 3, 81, 'A+'),
(3, 'AI301', 4, 89, 'A+'), (3, 'CS301', 4, 70, 'A'), (4, 'AI201', 3, 92, 'O'),
(4, 'AI301', 4, 76, 'A'), (4, 'AI401', 4, 84, 'A+'), (5, 'AI201', 3, 58, 'B'),
(5, 'AI301', 4, 63, 'B+'), (6, 'EC201', 3, 79, 'A'), (6, 'CS101', 3, 55, 'B'),
(6, 'EC301', 4, 82, 'A+'), (7, 'EC201', 3, 45, 'C'), (7, 'EC301', 4, 61, 'B+'),
(8, 'CS101', 3, 78, 'A'), (9, 'CS201', 3, 90, 'O'), (9, 'CS301', 4, 95, 'O'),
(9, 'CS401', 4, 88, 'A+');

```

Q3: Check the number of rows inserted in each table

```
SELECT 'department' AS table_name, COUNT(*) AS total_rows FROM department
UNION ALL SELECT 'instructor', COUNT(*) FROM instructor
UNION ALL SELECT 'student', COUNT(*) FROM student
UNION ALL SELECT 'student_phone', COUNT(*) FROM student_phone
UNION ALL SELECT 'course', COUNT(*) FROM course
UNION ALL SELECT 'enrollment', COUNT(*) FROM enrollment;
```

Q4: SELECT all columns, selected columns, aliases and computed columns

```
SELECT * FROM department;
SELECT roll_no AS `Roll No`, UPPER(student_name) AS `Name`,
       TIMESTAMPDIFF(YEAR, dob, '2026-10-09') AS `Age`
FROM student;
SELECT DISTINCT city FROM student;
```

Q5: WHERE with comparison and logical operators (AND, OR, NOT)

```
SELECT roll_no, student_name, city, dept_id
FROM student
WHERE city = 'Pune' AND dept_id = 20;

SELECT instr_name, designation, salary
FROM instructor
WHERE salary > 150000 OR designation = 'Assoc. Prof';

SELECT course_id, title, credits FROM course WHERE NOT dept_id = 10;
```

Q6: Special operators: BETWEEN, IN, LIKE and IS NULL

```
SELECT student_id, course_id, marks FROM enrollment
WHERE marks BETWEEN 80 AND 90;

SELECT roll_no, student_name, city FROM student
WHERE city IN ('Mumbai', 'Nashik', 'Nagpur');

SELECT roll_no, student_name FROM student
WHERE student_name LIKE 'A%' OR roll_no LIKE 'S24_I%';

SELECT roll_no, student_name, city FROM student WHERE city IS NULL;
SELECT course_id, title FROM course WHERE instr_id IS NULL;
```

Q7: ORDER BY (ascending, descending, multiple columns) and LIMIT

```
SELECT roll_no, student_name, dept_id, dob
FROM student
ORDER BY dept_id ASC, dob DESC;

SELECT student_id, course_id, marks
FROM enrollment
ORDER BY marks DESC
LIMIT 5;

SELECT instr_id, instr_name, salary
FROM instructor
ORDER BY salary DESC
LIMIT 3 OFFSET 3;
```

LIMIT 3 OFFSET 3 skips the first three rows and returns the next three (page 2 of a list shown three rows per page). The equivalent MySQL form is LIMIT 3, 3.

Q8: UPDATE: single row, multiple rows, expressions and CASE

```

UPDATE student SET city = 'Satara' WHERE roll_no = 'S24CS003';

SELECT instr_name, salary FROM instructor WHERE designation = 'Asst. Prof';
UPDATE instructor SET salary = salary * 1.10 WHERE designation = 'Asst. Prof';
SELECT instr_name, salary FROM instructor WHERE designation = 'Asst. Prof';

UPDATE enrollment SET marks = marks + 5 WHERE course_id = 'EC201' AND marks < 50;
UPDATE enrollment
SET    grade = CASE
                WHEN marks >= 90 THEN 'O'  WHEN marks >= 80 THEN 'A+'
                WHEN marks >= 70 THEN 'A'  WHEN marks >= 60 THEN 'B+'
                WHEN marks >= 50 THEN 'B'  WHEN marks >= 40 THEN 'C'
                ELSE 'F'
            END
WHERE course_id = 'EC201';
SELECT * FROM enrollment WHERE course_id = 'EC201';

```

Q9: DELETE: specific rows and cascading delete

```

DELETE FROM student_phone WHERE phone = '9876500012';
SELECT * FROM student_phone WHERE student_id = 1;

SELECT COUNT(*) AS enrollments_of_student_7 FROM enrollment WHERE student_id = 7;
DELETE FROM student WHERE roll_no = 'S24EC002';
SELECT COUNT(*) AS enrollments_of_student_7 FROM enrollment WHERE student_id = 7;

DELETE FROM course WHERE course_id = 'CS101';
SELECT COUNT(*) AS students_left FROM student;

```

Deleting student S24EC002 (student_id 7) also removes the student's enrollments because of ON DELETE CASCADE, whereas deleting course CS101 is refused because enrollments and other courses still refer to it.

Q10: Final state of the student table

```

SELECT student_id, roll_no, student_name, city
FROM    student
ORDER BY student_id;

```

8. Sample Input / Database Tables

The following tables show the data inserted by Q2 (read from the database after executing Q1 and Q2). The grade column follows the scale below.

Marks	90-100	80-89	70-79	60-69	50-59	40-49	Below 40
Grade	O	A+	A	B+	B	C	F

Table: department

dept_id	dept_code	dept_name	building	budget
10	CSE	Computer Engineering	A Block	2500000.00
20	AIML	AI and Machine Learning	B Block	3000000.00
30	ECE	Electronics and Telecomm	C Block	1800000.00
40	MECH	Mechanical Engineering	D Block	1500000.00
50	CIVIL	Civil Engineering	D Block	1200000.00

Table: instructor

instr_id	instr_name	designation	dept_id	salary	email	mentor_id
101	Anil Kulkarni	Professor	10	185000.00	anil.k@example.com	NULL
102	Meera Joshi	Professor	20	172000.00	meera.j@example.com	101
103	Rahul Patil	Asst. Prof	10	82000.00	rahul.p@example.com	101
104	Sneha Deshmukh	Assoc. Prof	20	115000.00	sneha.d@example.com	102
105	Vikram Shinde	Professor	30	168000.00	vikram.s@example.com	101
106	Pooja Kale	Asst. Prof	20	78000.00	pooja.k@example.com	104
107	Nitin Pawar	Asst. Prof	50	76000.00	nitin.p@example.com	105
108	Kavita More	Assoc. Prof	40	112000.00	kavita.m@example.com	105

Table: student

student_id	roll_no	student_name	gender	dob	dept_id	city	email
1	S24CS001	Aarav Sharma	M	2005-03-14	10	Pune	aarav@example.com
2	S24CS002	Isha Gupta	F	2005-07-22	10	Mumbai	isha@example.com
3	S24AI001	Rohan Mehta	M	2005-01-09	20	Pune	rohan@example.com
4	S24AI002	Priya Nair	F	2004-11-30	20	Nashik	priya@example.com
5	S24AI003	Kunal Verma	M	2005-05-18	20	Pune	kunal@example.com
6	S24EC001	Sneha Iyer	F	2005-09-02	30	Nagpur	sneha@example.com
7	S24EC002	Aditya Rao	M	2004-12-25	30	Mumbai	aditya@example.com
8	S24ME001	Neha Kulkarni	F	2005-02-11	40	Pune	neha@example.com
9	S24CS003	Yash Jadhav	M	2005-08-05	10	NULL	yash@example.com
10	S24AI004	Ananya Desai	F	2005-04-27	20	Mumbai	ananya@example.com

Table: student_phone

student_id	phone
1	9876500011
1	9876500012
2	9876500021
3	9876500031
4	9876500041
4	9876500042
6	9876500061
9	9876500091

Table: course

course_id	title	dept_id	credits	instr_id	prereq_id
AI201	Python for Data Science	20	3	104	CS101
AI301	Machine Learning	20	4	102	AI201
AI401	Deep Learning	20	3	106	AI301
CS101	Programming in C	10	4	103	NULL
CS201	Data Structures	10	4	103	CS101
CS301	Database Management Systems	10	3	101	CS201
CS401	Advanced Databases	10	3	101	CS301
EC201	Digital Electronics	30	3	105	NULL
EC301	Signals and Systems	30	4	NULL	EC201
ME201	Engineering Mechanics	40	3	108	NULL

Table: enrollment (25 rows)

student_id	course_id	semester	marks	grade
1	AI201	3	72	A
1	CS101	3	78	A
1	CS201	3	85	A+
1	CS301	4	91	O
2	CS101	3	88	A+
2	CS201	3	67	B+
2	CS301	4	74	A
3	AI201	3	81	A+
3	AI301	4	89	A+
3	CS101	3	65	B+
3	CS301	4	70	A
4	AI201	3	92	O
4	AI301	4	76	A
4	AI401	4	84	A+
5	AI201	3	58	B
5	AI301	4	63	B+
6	CS101	3	55	B
6	EC201	3	79	A
6	EC301	4	82	A+
7	EC201	3	45	C
7	EC301	4	61	B+
8	CS101	3	78	A
9	CS201	3	90	O
9	CS301	4	95	O
9	CS401	4	88	A+

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine.

Output of Q1: Create the database and tables (schema of Practical 2)

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.00 sec)

mysql> CREATE TABLE department ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE instructor ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student_phone ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course ( ...;
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE enrollment ( ...;
Query OK, 0 rows affected (0.02 sec)
```

Output of Q2: INSERT: single row, with column list, and multiple rows

```
mysql> INSERT INTO department VALUES (10, 'CSE', 'Computer Engineering', 'A Block', 2500000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES ...;
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_phone VALUES ...;
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course VALUES ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES ...;
Query OK, 25 rows affected (0.01 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Output of Q3: Check the number of rows inserted in each table

```
mysql> SELECT 'department' AS table_name, COUNT(*) AS total_rows FROM department
-> UNION ALL SELECT 'instructor', COUNT(*) FROM instructor
-> UNION ALL SELECT 'student', COUNT(*) FROM student
-> UNION ALL SELECT 'student_phone', COUNT(*) FROM student_phone
-> UNION ALL SELECT 'course', COUNT(*) FROM course
-> UNION ALL SELECT 'enrollment', COUNT(*) FROM enrollment;
```

table_name	total_rows
department	5
instructor	8
student	10
student_phone	8
course	10
enrollment	25

6 rows in set (0.00 sec)

Output of Q4: SELECT all columns, selected columns, aliases and computed columns

```
mysql> SELECT * FROM department;
```

dept_id	dept_code	dept_name	building	budget
10	CSE	Computer Engineering	A Block	2500000.00
20	AIML	AI and Machine Learning	B Block	3000000.00
30	ECE	Electronics and Telecomm	C Block	1800000.00
40	MECH	Mechanical Engineering	D Block	1500000.00
50	CIVIL	Civil Engineering	D Block	1200000.00

5 rows in set (0.00 sec)

```
mysql> SELECT roll_no AS `Roll No`, UPPER(student_name) AS `Name`,
-> TIMESTAMPDIFF(YEAR, dob, '2026-10-09') AS `Age`
-> FROM student;
```

Roll No	Name	Age
S24CS001	AARAV SHARMA	21
S24CS002	ISHA GUPTA	21
S24AI001	ROHAN MEHTA	21
S24AI002	PRIYA NAIR	21
S24AI003	KUNAL VERMA	21
S24EC001	SNEHA IYER	21
S24EC002	ADITYA RAO	21
S24ME001	NEHA KULKARNI	21
S24CS003	YASH JADHAV	21
S24AI004	ANANYA DESAI	21

10 rows in set (0.00 sec)

```
mysql> SELECT DISTINCT city FROM student;
```

city
Pune
Mumbai
Nashik
Nagpur
NULL

5 rows in set (0.00 sec)

Output of Q5: WHERE with comparison and logical operators (AND, OR, NOT)

```
mysql> SELECT roll_no, student_name, city, dept_id
-> FROM student
-> WHERE city = 'Pune' AND dept_id = 20;
+-----+-----+-----+-----+
| roll_no | student_name | city | dept_id |
+-----+-----+-----+-----+
| S24AI001 | Rohan Mehta | Pune | 20 |
| S24AI003 | Kunal Verma | Pune | 20 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT instr_name, designation, salary
-> FROM instructor
-> WHERE salary > 150000 OR designation = 'Assoc. Prof';
+-----+-----+-----+
| instr_name | designation | salary |
+-----+-----+-----+
| Anil Kulkarni | Professor | 185000.00 |
| Meera Joshi | Professor | 172000.00 |
| Sneha Deshmukh | Assoc. Prof | 115000.00 |
| Vikram Shinde | Professor | 168000.00 |
| Kavita More | Assoc. Prof | 112000.00 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT course_id, title, credits FROM course WHERE NOT dept_id = 10;
+-----+-----+-----+
| course_id | title | credits |
+-----+-----+-----+
| AI201 | Python for Data Science | 3 |
| AI301 | Machine Learning | 4 |
| AI401 | Deep Learning | 3 |
| EC201 | Digital Electronics | 3 |
| EC301 | Signals and Systems | 4 |
| ME201 | Engineering Mechanics | 3 |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

Output of Q6: Special operators: BETWEEN, IN, LIKE and IS NULL

```
mysql> SELECT student_id, course_id, marks FROM enrollment
-> WHERE marks BETWEEN 80 AND 90;
+-----+-----+-----+
| student_id | course_id | marks |
+-----+-----+-----+
| 1 | CS201 | 85 |
| 2 | CS101 | 88 |
| 3 | AI201 | 81 |
| 3 | AI301 | 89 |
| 4 | AI401 | 84 |
| 6 | EC301 | 82 |
| 9 | CS201 | 90 |
| 9 | CS401 | 88 |
+-----+-----+-----+
8 rows in set (0.00 sec)
```

```
mysql> SELECT roll_no, student_name, city FROM student
-> WHERE city IN ('Mumbai', 'Nashik', 'Nagpur');
+-----+-----+-----+
| roll_no | student_name | city |
+-----+-----+-----+
| S24CS002 | Isha Gupta | Mumbai |
| S24AI002 | Priya Nair | Nashik |
| S24EC001 | Sneha Iyer | Nagpur |
| S24EC002 | Aditya Rao | Mumbai |
| S24AI004 | Ananya Desai | Mumbai |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT roll_no, student_name FROM student
-> WHERE student_name LIKE 'A%' OR roll_no LIKE 'S24_I%';
+-----+-----+
| roll_no | student_name |
+-----+-----+
| S24CS001 | Aarav Sharma |
| S24AI001 | Rohan Mehta  |
| S24AI002 | Priya Nair   |
| S24AI003 | Kunal Verma  |
| S24EC002 | Aditya Rao   |
| S24AI004 | Ananya Desai |
+-----+-----+
6 rows in set (0.00 sec)

mysql> SELECT roll_no, student_name, city FROM student WHERE city IS NULL;
+-----+-----+-----+
| roll_no | student_name | city |
+-----+-----+-----+
| S24CS003 | Yash Jadhav  | NULL |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT course_id, title FROM course WHERE instr_id IS NULL;
+-----+-----+
| course_id | title |
+-----+-----+
| EC301     | Signals and Systems |
+-----+-----+
1 row in set (0.00 sec)
```

Output of Q7: ORDER BY (ascending, descending, multiple columns) and LIMIT

```
mysql> SELECT roll_no, student_name, dept_id, dob
-> FROM student
-> ORDER BY dept_id ASC, dob DESC;
+-----+-----+-----+-----+
| roll_no | student_name | dept_id | dob |
+-----+-----+-----+-----+
| S24CS003 | Yash Jadhav | 10 | 2005-08-05 |
| S24CS002 | Isha Gupta | 10 | 2005-07-22 |
| S24CS001 | Aarav Sharma | 10 | 2005-03-14 |
| S24AI003 | Kunal Verma | 20 | 2005-05-18 |
| S24AI004 | Ananya Desai | 20 | 2005-04-27 |
| S24AI001 | Rohan Mehta | 20 | 2005-01-09 |
| S24AI002 | Priya Nair | 20 | 2004-11-30 |
| S24EC001 | Sneha Iyer | 30 | 2005-09-02 |
| S24EC002 | Aditya Rao | 30 | 2004-12-25 |
| S24ME001 | Neha Kulkarni | 40 | 2005-02-11 |
+-----+-----+-----+-----+
10 rows in set (0.00 sec)

mysql> SELECT student_id, course_id, marks
-> FROM enrollment
-> ORDER BY marks DESC
-> LIMIT 5;
+-----+-----+-----+
| student_id | course_id | marks |
+-----+-----+-----+
| 9 | CS301 | 95 |
| 4 | AI201 | 92 |
| 1 | CS301 | 91 |
| 9 | CS201 | 90 |
| 3 | AI301 | 89 |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT instr_id, instr_name, salary
-> FROM instructor
-> ORDER BY salary DESC
-> LIMIT 3 OFFSET 3;
+-----+-----+-----+
| instr_id | instr_name | salary |
+-----+-----+-----+
```

```

+-----+-----+-----+
|      104 | Sneha Deshmukh | 115000.00 |
|      108 | Kavita More    | 112000.00 |
|      103 | Rahul Patil    | 82000.00  |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

Output of Q8: UPDATE: single row, multiple rows, expressions and CASE

```

mysql> UPDATE student SET city = 'Satara' WHERE roll_no = 'S24CS003';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT instr_name, salary FROM instructor WHERE designation = 'Asst. Prof';
+-----+-----+
| instr_name | salary |
+-----+-----+
| Rahul Patil | 82000.00 |
| Pooja Kale  | 78000.00 |
| Nitin Pawar | 76000.00 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> UPDATE instructor SET salary = salary * 1.10 WHERE designation = 'Asst. Prof';
Query OK, 3 rows affected (0.00 sec)
Rows matched: 3  Changed: 3  Warnings: 0

mysql> SELECT instr_name, salary FROM instructor WHERE designation = 'Asst. Prof';
+-----+-----+
| instr_name | salary |
+-----+-----+
| Rahul Patil | 90200.00 |
| Pooja Kale  | 85800.00 |
| Nitin Pawar | 83600.00 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> UPDATE enrollment SET marks = marks + 5 WHERE course_id = 'EC201' AND marks < 50;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE enrollment
-> SET      grade = CASE
->                WHEN marks >= 90 THEN 'O'  WHEN marks >= 80 THEN 'A+'
->                WHEN marks >= 70 THEN 'A'  WHEN marks >= 60 THEN 'B+'
->                WHEN marks >= 50 THEN 'B'  WHEN marks >= 40 THEN 'C'
->                ELSE 'F'
->            END
-> WHERE course_id = 'EC201';
Query OK, 1 row affected (0.00 sec)
Rows matched: 2  Changed: 1  Warnings: 0

mysql> SELECT * FROM enrollment WHERE course_id = 'EC201';
+-----+-----+-----+-----+-----+
| student_id | course_id | semester | marks | grade |
+-----+-----+-----+-----+-----+
|          6 | EC201    |         3 |    79 | A     |
|          7 | EC201    |         3 |    50 | B     |
+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

```

Output of Q9: DELETE: specific rows and cascading delete

```

mysql> DELETE FROM student_phone WHERE phone = '9876500012';
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM student_phone WHERE student_id = 1;
+-----+-----+
| student_id | phone      |
+-----+-----+
|          1 | 9876500011 |
+-----+-----+

```

```

+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT COUNT(*) AS enrollments_of_student_7 FROM enrollment WHERE student_id = 7;
+-----+-----+
| enrollments_of_student_7 |
+-----+-----+
| 2 |
+-----+-----+
1 row in set (0.00 sec)

mysql> DELETE FROM student WHERE roll_no = 'S24EC002';
Query OK, 1 row affected (0.00 sec)

mysql> SELECT COUNT(*) AS enrollments_of_student_7 FROM enrollment WHERE student_id = 7;
+-----+-----+
| enrollments_of_student_7 |
+-----+-----+
| 0 |
+-----+-----+
1 row in set (0.00 sec)

mysql> DELETE FROM course WHERE course_id = 'CS101';
ERROR 1451 (23000) at line 217: Cannot delete or update a parent row: a foreign key
constraint fails (`college_db`.`course`, CONSTRAINT `fk_course_prereq` FOREIGN KEY
(`prereq_id`) REFERENCES `course` (`course_id`))

mysql> SELECT COUNT(*) AS students_left FROM student;
+-----+-----+
| students_left |
+-----+-----+
| 9 |
+-----+-----+
1 row in set (0.00 sec)

```

Output of Q10: Final state of the student table

```

mysql> SELECT student_id, roll_no, student_name, city
-> FROM student
-> ORDER BY student_id;
+-----+-----+-----+-----+
| student_id | roll_no | student_name | city |
+-----+-----+-----+-----+
| 1 | S24CS001 | Aarav Sharma | Pune |
| 2 | S24CS002 | Isha Gupta | Mumbai |
| 3 | S24AI001 | Rohan Mehta | Pune |
| 4 | S24AI002 | Priya Nair | Nashik |
| 5 | S24AI003 | Kunal Verma | Pune |
| 6 | S24EC001 | Sneha Iyer | Nagpur |
| 8 | S24ME001 | Neha Kulkarni | Pune |
| 9 | S24CS003 | Yash Jadhav | Satara |
| 10 | S24AI004 | Ananya Desai | Mumbai |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)

```

10. Result

The tables of *college_db* were populated with 66 rows using different forms of INSERT. Data was retrieved using SELECT with aliases, expressions, DISTINCT, WHERE (comparison, logical, BETWEEN, IN, LIKE, IS NULL), ORDER BY and LIMIT/OFFSET. UPDATE statements changed the city of 1 student, the salary of 3 instructors and the marks and grade of 1 enrollment as intended, and DELETE removed the selected rows, cascading to the enrollments of the deleted student, while the delete of a referenced course was correctly rejected.

11. Conclusion

DML statements are used for day-to-day work on the data stored in a database. The WHERE clause determines exactly which rows a SELECT, UPDATE or DELETE affects, so it must be written carefully - an UPDATE or DELETE without WHERE changes every row. ORDER BY is the only way to guarantee the order of results, and LIMIT/OFFSET is useful for top-N lists and pagination. The constraints defined in Practical 2 continued to protect the data during DML operations.